SIMATS ENGINEERING
ASSESSMENT TOOL 2
Case study

COURSE NAME: OBJECT ORIENTED ANALYSIS AND DESIGN
COURSE CODE: CSA1104

COURSE OUTCOME COVERED

CO3: Analyze system requirements using object-oriented techniques to identify classes, attributes, and relationships and develop use case models**.** *(BL4)*

Assessment Tool Weightage: 40%

Code of Conduct:

I ***Durga Sree R/192411189*** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

Case study

QUESTIONS: Any 4 **Total Marks:40**

1. Online Hotel Reservation System

A travel company plans to develop an online hotel reservation platform where customers can search hotels, book rooms, make payments, and provide reviews. Hotel managers can update room availability and pricing, while administrators manage users and transactions.

Question:

Analyze the system and explain how object-oriented principles (encapsulation, inheritance, polymorphism) can be applied. Also, suggest a suitable SDLC model and justify your choice for this system.

Object-Oriented Principles

The Online Hotel Reservation System can be developed effectively using object-oriented concepts.

1. Encapsulation

Encapsulation combines data and methods within a class and protects data from direct access.

Classes:

- Customer – customerId, name, email, phone
- Hotel – hotelId, hotelName, location
- Room – roomNo, roomType, price, availability
- Booking – bookingId, checkIn, checkOut, status
- Payment – paymentId, amount, paymentStatus
- Review – reviewId, rating, comments

For example, room availability and booking details can be kept private and accessed through methods such as checkAvailability() and updateAvailability().

2. Inheritance

Common properties can be inherited from a parent class.

Example:

User

- userId
- name

- email
- login()

Subclasses:

- Customer
- HotelManager
- Administrator

This avoids duplication and improves code reuse.

3. Polymorphism

The same operation can behave differently for different classes.

For example, a manageAccount() method can have different implementations:

- Customer → update personal details
- HotelManager → update hotel and room details
- Administrator → manage users and transactions

Relationships

- Customer **makes** Booking
- Booking **contains** Room
- Customer **makes** Payment
- Customer **writes** Review
- HotelManager **manages** Hotel
- Administrator **manages** Users and Transactions

Suitable SDLC Model – Agile Model

Agile is suitable because hotel requirements may change based on customer and hotel-manager feedback.

Reasons:

1. Development can be divided into small iterations.
2. Booking, payment, review, and hotel management can be developed separately.
3. Frequent customer feedback can be incorporated.
4. Errors can be identified early.
5. New features such as coupons, cancellation, and notifications can be added easily.

Conclusion

Object-oriented principles provide modularity, reusability, security, and flexibility. Agile supports continuous improvement and changing requirements, making it suitable for the Online Hotel Reservation System.

2. University Management System

A university intends to automate its operations including student admissions, course registration, examination management, and result processing. Different users such as students, faculty members, examination staff, and administrators interact with the system.

Question:

Design a structured approach using object-oriented concepts to model this system. Identify key classes and relationships, and recommend an SDLC model for efficient development with reasoning.

Object-Oriented Model

The University Management System can be represented using classes corresponding to major entities.

Major Classes

Student

- studentId
- name
- department
- email
- registerCourse()
- viewResult()

Faculty

- facultyId
- name
- department
- teachCourse()
- enterMarks()

Course

- courseId
- courseName
- credits
- registerStudent()

Examination

- examId
- examDate
- conductExam()

Result

- resultId
- marks
- grade
- calculateGrade()

Administrator

- adminId
- name
- manageUsers()

Relationships

- Student **registers for** Course
- Faculty **teaches** Course
- Student **takes** Examination
- Examination **produces** Result
- Faculty **updates** Result
- Administrator **manages** Students, Faculty and Courses

Object-Oriented Concepts

Encapsulation:
Student details, marks, and examination information are kept inside their respective classes and accessed through methods.

Inheritance:
A general User class can be created.

User → Student
User → Faculty
User → Administrator

Common attributes such as userId, name, email, and login() can be reused.

Polymorphism:
A method such as viewDetails() can behave differently for Student, Faculty, and Administrator.

Suitable SDLC Model – Agile Model

Agile is suitable because university requirements can change frequently.

Reasons:

1. Modules can be developed incrementally.
2. Admission, registration, examination, and results can be developed as separate iterations.
3. Faculty and student feedback can be obtained regularly.
4. Testing can be performed continuously.
5. New university policies can be incorporated easily.

Conclusion

Object-oriented modeling clearly represents university entities and their relationships. Agile provides flexibility and allows the system to be developed module by module.

3. Online Retail Shopping Platform

A retail company wants to develop an e-commerce platform where customers can browse products, place orders, track deliveries, and make online payments. Vendors manage product catalogs, while administrators monitor transactions and inventory.

Question:

Discuss how modular software design can be achieved using object-oriented techniques. Evaluate different SDLC models and select the most appropriate one for this scenario with justification.

Modular Software Design Using OOP

The e-commerce system can be divided into independent modules using object-oriented design.

Major Modules

1. **User Management**
2. **Product Management**
3. **Shopping Cart**
4. **Order Management**
5. **Payment Management**
6. **Delivery Tracking**
7. **Inventory Management**

Major Classes

Customer

- customerId
- name
- address
- browseProduct()
- placeOrder()

Product

- productId
- name
- price
- quantity
- displayProduct()

Order

- orderId
- orderDate
- status
- createOrder()

Payment

- paymentId
- amount
- status
- processPayment()

Vendor

- vendorId
- name
- manageProducts()

Administrator

- adminId
- monitorTransactions()
- manageInventory()

Object-Oriented Techniques

Encapsulation:
Product price, inventory quantity, payment details, and customer information are protected inside their classes.

Inheritance:
A common User class can have subclasses:

User → Customer
User → Vendor
User → Administrator

Polymorphism:
Different payment methods can implement a common makePayment() method.

Example:

Payment → CreditCardPayment
Payment → UPI_Payment
Payment → NetBankingPayment

Each class performs payment differently.

SDLC Models Evaluation

Waterfall:
Simple and structured but not suitable when requirements change frequently.

Spiral:
Good for risk management but can be expensive and complex.

Agile:
Supports frequent changes, continuous testing, and customer feedback.

Selected Model – Agile

Agile is the most appropriate because e-commerce requirements change frequently.

Advantages:

- Faster development
- Continuous customer feedback
- Easy addition of new payment methods
- Continuous testing
- Frequent releases
- Easy modification of product and delivery features

Conclusion

Object-oriented modular design makes the e-commerce system reusable, maintainable, and scalable. Agile is suitable because it supports changing business and customer requirements.

4. Smart Waste Management System

A municipality introduces a smart waste management system where IoT-enabled bins monitor waste levels and automatically notify collection vehicles when bins are full. Citizens can also report waste-related issues through a mobile application.

Question:

Apply object-oriented system development principles to design this system. Identify the life cycle model best suited for handling real-time data, sensor integration, and evolving requirements, and explain why.

Object-Oriented System Design

The Smart Waste Management System combines software with IoT sensors and mobile applications.

Major Classes

SmartBin

- binId
- location
- wasteLevel
- sensorStatus
- measureWasteLevel()
- sendAlert()

Sensor

- sensorId
- sensorType
- readData()

CollectionVehicle

- vehicleId

- driverName
- location
- collectWaste()

Citizen

- citizenId
- name
- contact
- reportIssue()

Notification

- notificationId
- message
- sendNotification()

Administrator

- adminId
- monitorBins()
- generateReport()

Relationships

- SmartBin **contains** Sensor
- Sensor **monitors** SmartBin
- SmartBin **sends** Notification
- CollectionVehicle **collects waste from** SmartBin
- Citizen **reports** waste issues
- Administrator **monitors** the complete system

Object-Oriented Principles

Encapsulation:
Sensor readings and bin status are protected inside the SmartBin and Sensor classes.

Inheritance:
A general Sensor class can be created.

Sensor → UltrasonicSensor
Sensor → WeightSensor
Sensor → GasSensor

Polymorphism:
All sensors can use a common readData() method, but each sensor reads different types of data.

Suitable Life Cycle Model – Agile Model

Agile is suitable because sensor requirements and real-time monitoring requirements can evolve during development.

Reasons:

1. IoT hardware and software can be integrated incrementally.
2. Sensor-related problems can be identified early.
3. Real-time features can be tested continuously.
4. Mobile application features can be added in iterations.
5. Municipality feedback can be incorporated.
6. New sensors can be integrated easily.

Conclusion

Object-oriented design provides modularity and easy sensor integration. Agile is suitable because the system has evolving requirements, real-time data, and continuous hardware-software integration.

RUBRICS FOR CALCULATION:

Case study

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	25	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	25	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	20	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	20	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	10	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand